

Shared-State Containers and Runtime References

Refactor specification for EDSL multi-agent simulations

Expected Parrot

August 2026

Contents

1	Status	3
2	Problem statement	4
3	Design principles	5
3.1	One object, one cardinality	5
3.2	Python syntax, declarative meaning	5
3.3	Explicit creation and routing	5
3.4	Validate before model calls	5
3.5	Preserve scientific choices	5
3.6	Minimize changes to core EDSL objects	6
4	Proposed object model	7
4.1	SharedState: one world	7
4.2	SharedStateMap: many equivalent worlds	7
4.3	Mapping semantics	8
4.4	Homogeneous configuration	9
5	Stores and addressing	10
5.1	Store ownership	10
5.2	Store interface	10
5.3	Local store modes	10
5.4	Remote stores	11
6	Runtime references	12
6.1	current	12
6.2	Question answers	12
6.3	Stem	13
7	Routing interviews to states	14
7.1	One fixed state	14
7.2	A routed state map	14
7.3	Why routing is not <code>__getitem__</code>	15
7.4	Routing validation	15
8	Class-based schedules	16
9	Primitive integration	17
9.1	Operations remain anchored survey steps	17

9.2	Before-question operations	17
9.3	Visibility	18
10	Closing and lifecycle	19
11	Reading and analysis	20
12	Serialization	21
12.1	SharedState	21
12.2	SharedStateMap	21
12.3	Router	21
13	Concurrency and consistency	23
14	Errors	24
15	Backward compatibility	25
15.1	Deprecation path	25
15.2	Read API migration	25
15.3	String-reference migration	26
15.4	Schedule migration	26
16	Example rewrite template	27
17	Migration plan for the example suite	29
17.1	Phase 1: characterization	29
17.2	Phase 2: foundations	29
17.3	Phase 3: simple examples	29
17.4	Phase 4: scheduling and visibility	30
17.5	Phase 5: dynamic workflows	30
17.6	Phase 6: remove legacy syntax	30
18	Test specification	31
18.1	Unit tests	31
18.2	Concurrency tests	31
18.3	Serialization tests	31
18.4	Example equivalence tests	31
19	Acceptance criteria	33
20	Decisions and open questions	34
20.1	Decisions made by this specification	34
20.2	Questions to resolve during prototyping	34

Chapter 1

Status

This document specifies a redesign of experimental EDSL shared state. It is an implementation target, not documentation of the current branch.

The working collection name is **SharedStateMap**. The name can change before implementation, but the distinction between one state and a keyed collection of states is normative.

The redesign has four related goals:

1. Make **SharedState** mean exactly one shared world.
2. Represent many equivalent shared worlds with a separate mapping object.
3. Replace string-based runtime references with the serializable **current** namespace and explicit question-answer references.
4. Replace string-based interview schedules with validated schedule objects.

The eventual acceptance test is that every existing economic-game and shared-state example can be rewritten more clearly without adding game-specific orchestration to core **Survey** or **Question** classes.

Chapter 2

Problem statement

The current constructor combines several concepts:

```
SharedState(  
    scope="{ agent.pair_id }",  
    store=FileStateStore(path),  
    game=game,  
)
```

The `scope` argument may be:

- a concrete identifier;
- a template that selects an identifier at runtime;
- a default used by some reads and writes;
- an apparent declaration of how many worlds exist.

Meanwhile, `state.scopes()` returns identifiers discovered in the store, not values declared by the constructor. A single object therefore appears to be both one state and a collection of states.

The implementation model is actually simpler than the API suggests. The store is an event table keyed by (`scope`, `target`). Each scope contains the same configured primitives. Scopes can be created lazily when their first event is written.

The redesign should expose that model directly:

```
SharedStateMap  
+-- "pair-1" -> SharedState  
|               +-- game  
+-- "pair-2" -> SharedState  
|               +-- game  
+-- "pair-3" -> SharedState  
|               +-- game
```

Chapter 3

Design principles

3.1 One object, one cardinality

`SharedState` represents one scope. `SharedStateMap` represents zero or more scopes. Neither object changes apparent cardinality depending on the type of an argument.

3.2 Python syntax, declarative meaning

Authors use ordinary attribute access and typed objects:

```
current.agent.name  
offer.answer  
Serial()  
GroupedRoundRobin(...)
```

These objects compile to data. Arbitrary callbacks are not required for the normal path, so jobs remain serializable for remote execution.

3.3 Explicit creation and routing

State configuration, scope creation, and interview-to-scope routing are different operations. The API names them separately.

3.4 Validate before model calls

Invalid primitive names, scope keys, answer dependencies, role assignments, schedule fields, visibility reads, and store/configuration mismatches should fail during construction or preflight.

3.5 Preserve scientific choices

Concurrency, observation order, reveal boundaries, missing-player behavior, and settlement rules are part of the treatment. Convenience APIs must not choose them silently.

3.6 Minimize changes to core EDSL objects

The implementation should live primarily in `eds1.sharedstate`, runtime reference machinery, and scheduling. **Survey** needs only a general protocol for anchored executable steps and dependency validation. Questions need only an explicit answer reference and support for **Stem**; they should not acquire game-specific methods.

Chapter 4

Proposed object model

4.1 SharedState: one world

A `SharedState` has exactly one concrete key, one store binding, and one or more named primitives.

```
committee = SharedState(  
    key="committee-1",  
    store=FileStateStore.create("committee.jsonl"),  
    discussion=SharedMessageBoard(),  
    agenda=SharedAgenda(),  
)
```

Its methods never accept a `scope` argument:

```
committee.read()  
committee.history(target="discussion")  
committee.close()  
committee.records(target="agenda")  
committee.render_markdown()
```

Normative requirements:

- `key` is a nonempty, serializable string.
- A state cannot change its key after construction.
- Primitive names are unique valid Python identifiers.
- Reserved names are rejected at construction.
- A state may contain multiple primitive types.
- Reads and writes are automatically addressed to the state's key.
- Equality and serialization include the key and primitive configuration, but not merely the current snapshot.

4.2 SharedStateMap: many equivalent worlds

A `SharedStateMap` is a mapping from concrete string keys to `SharedState` instances. It also supplies the shared primitive configuration used when a new key is created.

```
games_by_pair = SharedStateMap(  
    store=FileStateStore.create("ultimatum.jsonl"),
```



```

    game=games.ultimatum(stake=100),
)

```

No scope is created by this constructor:

```

len(games_by_pair)      # 0
list(games_by_pair)     # []

```

Indexing returns a one-world `SharedState`. If the key is new, indexing creates and durably materializes it, following `defaultdict` semantics:

```

pair_1 = games_by_pair["pair-1"]

isinstance(pair_1, SharedState)  # True
pair_1.key                      # "pair-1"
pair_1.game                     # bound game primitive
pair_1.read()
pair_1.close()

"pair-1" in games_by_pair       # True

```

All returned states use the map's store and an immutable copy or binding of its primitive configuration.

4.3 Mapping semantics

`SharedStateMap` implements `collections.abc.Mapping[str, SharedState]`, not a fully mutable dictionary.

Required behavior:

```

states[key]                # get or lazily create a concrete state
states.get(key)            # get a state only if the key is realized
key in states              # whether the key is realized in persistent storage
iter(states)              # realized keys in first-event order
len(states)               # number of realized keys
states.keys()
states.values()
states.items()

```

Creation is observable and unsurprising to Python users familiar with `defaultdict`. `get()`, membership tests, and iteration never create keys; indexing does:

```

states.get("pair-9")       # None
"pair-9" in states         # False

pair_9 = states["pair-9"]  # create and return SharedState
"pair-9" in states         # True

```

Materialization writes a scope-creation record containing the configuration fingerprint; it does not fabricate a primitive action.

`__getitem__` rejects missing, empty, or non-string keys. The map does not support arbitrary `__setitem__` because assigning a separately configured state could violate the map's schema. It does

not support deletion in version 1; scopes can be closed or archived but their audit history remains durable.

4.4 Homogeneous configuration

All states in one map have the same named primitive configuration. This permits one survey and one remote job specification to operate over all scopes.

```
states = SharedStateMap(  
    store=store,  
    discussion=SharedMessageBoard(),  
    agenda=SharedAgenda(),  
)
```

Every key contains `discussion` and `agenda`. Their contents differ by key.

Heterogeneous worlds belong in separate maps. Version 1 does not allow:

```
states["pair-1"] = SharedState(game=ultimatum)  
states["market-1"] = SharedState(book=double_auction)
```

This restriction simplifies validation, serialization, remote reconstruction, and analysis.

Chapter 5

Stores and addressing

5.1 Store ownership

For one world, `SharedState` owns its store binding. For many worlds, `SharedStateMap` owns the store binding and passes it to returned state handles.

The logical event address is:

```
(state_key, primitive_target, version)
```

The public API uses `key`; the persistence protocol may retain `scope` as its wire-format field during migration.

5.2 Store interface

State containers depend on a store protocol rather than a local file class:

```
class StateStore(Protocol):
    def apply(self, operation: Operation) -> WriteResult: ...
    def read(self, key, config, context=None, at_version=None) -> Snapshot: ...
    def close(self, key) -> None: ...
    def keys(self) -> list[str]: ...
    def history(self, key=None, target=None) -> list[StateEvent]: ...
    def materialize(self, key, config_fingerprint) -> None: ...
```

The store must order events monotonically within each key. It need not impose a meaningful global order across keys.

5.3 Local store modes

Opening behavior is explicit:

```
FileStateStore.create(path)    # fail if path exists
FileStateStore.resume(path)   # verify stored configuration fingerprint
FileStateStore.replay(path)   # read-only
```

The default constructor should not silently choose between a new study and a resumed one.

5.4 Remote stores

The same containers support a remote implementation:

```
store = RemoteStateStore.connect(study="study-42")
```

Remote stores must provide:

- atomic per-key operations;
- idempotency keys for retries;
- authentication and per-study authorization;
- configuration-fingerprint verification;
- durable version ordering;
- reader-specific views or sufficient inputs to compute them safely;
- close/finalize transactions;
- paginated event-history retrieval.

A remote run cannot depend on a path on the submitting machine.

Chapter 6

Runtime references

6.1 current

`current` is the root of a serializable runtime-reference expression tree:

```
current.agent.name
current.agent.generosity
current.assignment.pair_id
current.assignment.role
current.run.round
current.state.game.offer
```

Attribute access builds references; it does not inspect live objects while the survey is being authored.

Minimum namespaces:

Namespace	Meaning
<code>current.agent</code>	Persona identity and declared traits
<code>current.assignment</code>	Group, role, seat, turn, and treatment metadata
<code>current.run</code>	Round, replication, seed, and other run context
<code>current.state</code>	Authorized view of the selected <code>SharedState</code>

References serialize to explicit data, for example:

```
{"type":"runtime_ref","root":"assignment","path":["pair_id"]}
```

Unknown namespaces and paths fail during preflight when schemas are available. Dynamic indexing requires an explicitly declared key set or is rejected.

6.2 Question answers

Every question exposes an explicit deferred answer reference:

```
offer.answer
decision.answer
```

Passing a question object as shorthand for its answer is deprecated. The explicit property distinguishes the question definition from its eventual value and enables dependency and type checking.

6.3 Stem

Dynamic question text uses checked placeholders and runtime bindings:

```
question_text=Stem(  
    "You are {name}. The current offer is {offer}.",  
    name=current.agent.name,  
    offer=current.state.game.offer,  
)
```

Static text may remain a string. Dynamic `question_text` should not embed Jinja expressions. `Stem` compiles to a serializable rendering plan and records its state-read dependencies.

Chapter 7

Routing interviews to states

Routing belongs to run configuration, not to `SharedState` construction.

7.1 One fixed state

```
committee = SharedState(  
    key="committee-1",  
    store=store,  
    discussion=SharedMessageBoard(),  
)  
  
run = survey.run(  
    agents=agents,  
    shared_state=committee,  
    interview_schedule=Serial(),  
)
```

Every interview receives `committee` as `current.state`.

7.2 A routed state map

```
games_by_pair = SharedStateMap(  
    store=store,  
    game=games.ultimatum(stake=100),  
)  
  
run = protocol.run(  
    agents=agents,  
    assignments=pairs,  
    shared_state=games_by_pair.by(current.assignment.pair_id),  
    interview_schedule=GroupedRoundRobin(  
        group_by=current.assignment.pair_id,  
        order_by=current.assignment.turn,  
        finalize_when=current.state.game.terminal,
```

```
    ),  
)
```

`.by(reference)` creates a serializable `StateRouter`. For each interview it:

1. resolves the reference to a concrete string key;
2. obtains `games_by_pair[key]`;
3. binds that one-world state as `current.state`;
4. executes reads and operations only against that state.

The map itself is never exposed as `current.state`; an interview always sees one selected `SharedState`.

7.3 Why routing is not `__getitem__`

This spelling is intentionally not used:

```
shared_state=games_by_pair[current.assignment.pair_id]
```

`Mapping.__getitem__` accepts a concrete key and returns a `SharedState`. Overloading it to accept a deferred expression would make the same operation return different types. `.by(...)` clearly constructs a router rather than performing dictionary lookup.

7.4 Routing validation

Preflight verifies:

- the routing expression resolves to `str`;
- every assigned interview has a routing value;
- keys are nonempty;
- grouping fields used by the schedule agree with state-routing fields when the mechanism requires one group per state;
- the selected map contains all primitives referenced by the protocol;
- remote credentials authorize the selected store.

Chapter 8

Class-based schedules

Schedules are immutable, serializable values. Strings such as "serial" are deprecated aliases at the public boundary.

```
Serial()
Concurrent(max_concurrency=20)
ConcurrentRound(
    group_by=current.assignment.group_id,
    snapshot=RoundStart,
    reveal=RoundEnd,
)
GroupedRoundRobin(
    group_by=current.assignment.pair_id,
    order_by=current.assignment.turn,
    finalize_when=current.state.game.terminal,
)
RepeatedRounds(
    count=4,
    group_by=current.assignment.group_id,
    within_round=ConcurrentRound(...),
)
```

Schedule objects define scientific semantics as well as execution strategy:

- ordering;
- grouping;
- snapshot/read watermark;
- reveal boundary;
- completion and close condition;
- retry behavior;
- maximum concurrency.

`max_concurrency` may affect throughput but must not change the logical order, visibility, priority, or settlement of a correctly specified mechanism.

Chapter 9

Primitive integration

9.1 Operations remain anchored survey steps

Primitives continue to produce declarative steps:

```
comment = QuestionFreeText(...)

survey = Survey([
    comment,
    states.comments.append(
        author=current.agent.name,
        text=comment.answer,
    ),
])
```

Here `states.comments` is the map's unbound primitive definition. The operation does not choose a key during survey construction. At runtime it applies to the one state selected by the run's `StateRouter`.

For a single `SharedState`, `state.comments.append(...)` produces the same operation definition but the key is already bound.

9.2 Before-question operations

```
survey = Survey([
    work_states.work.claim_before(
        review,
        claimant=current.agent.name,
    ),
    review,
    work_states.work.complete(
        claimant=current.agent.name,
        result=review.answer,
    ),
])
```

Before-question operations carry an idempotency key derived from run, interview, question, state key,

primitive target, and operation.

9.3 Visibility

`current.state` contains the authorized primitive view for the active agent, assignment, run stage, and version watermark. Storage and visibility are separate: a private value can be persisted for audit without appearing in a prompt.

`Stem` records read dependencies so preflight can reject unauthorized reads and the remote runner can reconstruct the same access plan.

Chapter 10

Closing and lifecycle

States progress through two observable conditions:

```
open -> closed
```

- **Open:** the state has been materialized by map lookup or constructed directly; writes are allowed.
- **Closed:** a durable close event exists; further writes are rejected.

Closing is idempotent:

```
states["pair-1"].close()  
states["pair-1"].close() # no additional effect
```

Closing invokes each primitive's declarative finalization in one logical transaction. If finalization fails, the state must not appear partially closed.

The map may provide bulk operations only with explicit keys:

```
states.close_many(["pair-1", "pair-2"])
```

There is no implicit `close_all()` in version 1 because newly created or failed groups may require different completion policies.

Chapter 11

Reading and analysis

One-state reads require no key argument:

```
pair = states["pair-1"]
pair.read()
pair.read(at_version=2)
pair.history(target="game")
pair.records(target="game")
```

Map-level analysis makes plurality explicit:

```
states.realized_keys()
states.snapshots()
states.records(target="game")
states.history(target="game")
```

Map-level rows always include `state_key`, `version`, and `closed`. The name `realized_keys()` is preferred to `scopes()` because it states that the result comes from persisted state rather than configuration.

Historical reads must reproduce the authorized view at that version, not apply today's visibility context silently. APIs accepting viewer context make it explicit:

```
pair.read(at_version=2, viewer=researcher_view)
```

Chapter 12

Serialization

12.1 SharedState

Illustrative representation:

```
{
  "type": "shared_state",
  "version": 1,
  "key": "committee-1",
  "store": {"type": "file", "path": "committee.jsonl", "mode": "resume"},
  "primitives": {
    "discussion": {"type": "message_board"},
    "agenda": {"type": "agenda"}
  }
}
```

12.2 SharedStateMap

```
{
  "type": "shared_state_map",
  "version": 1,
  "store": {"type": "remote", "study": "study-42"},
  "primitives": {
    "game": {"type": "configured_game", "configuration": {}}
  }
}
```

Realized keys and events live in the store; they are not duplicated in the map configuration. A portable run package may include an exported store snapshot.

12.3 Router

```
{
  "type": "state_router",
```

```
"map": {"ref": "shared-state-map-1"},
"key_by": {
  "type": "runtime_ref",
  "root": "assignment",
  "path": ["pair_id"]
}
}
```

Serialization validates that every primitive, reference, schedule, condition, and expression has a registered data representation. Arbitrary callables mark a job local-only and fail remote preflight.

Chapter 13

Concurrency and consistency

The redesign does not treat file locking as the scientific concurrency model. The scheduler defines logical behavior; the store enforces atomicity.

Minimum guarantees:

1. An operation is atomic within one state key.
2. Versions increase monotonically within that key.
3. Repeating an idempotency key has no additional effect.
4. A closed state rejects writes.
5. A simultaneous round can read a declared common version watermark even as writes are persisted.
6. Cross-key execution may proceed concurrently without changing within-key semantics.

Operations spanning multiple state keys are excluded from version 1. Markets or matching mechanisms requiring global settlement should use one state whose primitive contains all participants, rather than attempt an uncoordinated transaction across map entries.

Chapter 14

Errors

Errors should name the object, key, target, and corrective action.

```
StateKeyError: SharedStateMap key must be a nonempty string; received ''.
```

```
StateConfigurationError: store contains fingerprint 91ab... for key 'pair-2',  
but this map has fingerprint 73cd.... Resume with the original configuration  
or create a new store.
```

```
StateRoutingError: agent 'Amina' has no assignment field 'pair_id' required by  
games_by_pair.by(current.assignment.pair_id).
```

```
StateRoutingError: schedule groups by 'team_id' but shared state routes by  
'pair_id'; ultimatum requires one state per scheduled group.
```

```
StateClosedError: cannot apply game.respond to closed state 'pair-7'.
```

```
StateDependencyError: operation comments.append refers to comment.answer, but  
question 'comment' does not precede the operation in this protocol.
```

Chapter 15

Backward compatibility

15.1 Deprecation path

Current code:

```
SharedState(  
  scope="{ { agent.pair_id } }",  
  store=store,  
  game=game,  
)
```

becomes:

```
states = SharedStateMap(store=store, game=game)  
router = states.by(current.agent.pair_id)
```

The compatibility constructor can translate:

- a literal `scope="committee-1"` into a one-world `SharedState(key=...)`;
- a whole-trait template into `SharedStateMap(...).by(current.agent.<trait>)`.

It should emit a targeted deprecation warning containing the equivalent new code. General Jinja scope expressions that cannot be translated safely should fail with a migration message.

15.2 Read API migration

```
# Before  
state.read(scope="pair-1")  
state.history(scope="pair-1", target="game")  
state.scopes()  
  
# After  
states["pair-1"].read()  
states["pair-1"].history(target="game")  
states.realized_keys()
```

15.3 String-reference migration

```
# Before  
player="{ { agent.name } }"  
  
# After  
player=current.agent.name
```

15.4 Schedule migration

```
# Before  
.run(interview_schedule="serial")  
  
# After  
.run(interview_schedule=Serial())
```

Chapter 16

Example rewrite template

Every example should follow the same visible structure:

```
def personas() -> AgentList:
  ...

def assignments(agents):
  ...

def state_map(store) -> SharedStateMap:
  return SharedStateMap(
    store=store,
    game=games.some_mechanism(...),
  )

def protocol(states) -> RoleProtocol:
  action = QuestionMultipleChoice(
    question_name="action",
    question_text=Stem(
      "Current state: {state}. Choose an action.",
      state=current.state.game,
    ),
    question_options=[...],
  )
  return RoleProtocol({
    "role": Survey([
      action,
      states.game.act(
        player=current.agent.name,
        action=action.answer,
      ),
    ]),
  })
```

```

def schedule(states):
    return GroupedRoundRobin(
        group_by=current.assignment.group_id,
        order_by=current.assignment.turn,
        finalize_when=current.state.game.terminal,
    )

def run(path, model):
    agents = personas()
    design = assignments(agents)
    states = state_map(FileStateStore.create(path))
    return protocol(states).run(
        agents=agents,
        assignments=design,
        model=model,
        shared_state=states.by(current.assignment.group_id),
        interview_schedule=schedule(states),
    )

```

The repeated shape is intentional. If all examples require identical plumbing, the next abstraction should remove it without hiding `personas`, `assignments`, `mechanism`, `protocol`, `routing`, or `schedule`.

Chapter 17

Migration plan for the example suite

17.1 Phase 1: characterization

Before changing APIs:

- freeze tests for event histories and final records;
- classify each example as single-state or mapped-state;
- record grouping, routing, schedule, visibility, and completion policies;
- identify examples that currently rely on response completion order;
- identify local-only callables and dynamic Jinja reads.

17.2 Phase 2: foundations

Implement and test:

1. Runtime reference expression objects and `current`.
2. `Question.answer`.
3. `Stem` compilation and dependency metadata.
4. Schedule classes and string aliases.
5. One-world `SharedState` without scope parameters on read methods.
6. `SharedStateMap` mapping and router.
7. Store create/resume/replay modes and fingerprints.

17.3 Phase 3: simple examples

Rewrite examples that exercise one primitive and simple scheduling:

- logs and message boards;
- work pools;
- forecasts;
- one-shot matrix games;
- ultimatum games.

Use these to stabilize naming, error messages, serialization, and result APIs.

17.4 Phase 4: scheduling and visibility

Rewrite:

- repeated games;
- simultaneous public-goods games;
- information cascades;
- private signaling games;
- live and sealed auctions;
- multi-stage voting and deliberation.

These examples validate snapshots, reveal boundaries, private views, and terminal conditions.

17.5 Phase 5: dynamic workflows

Rewrite:

- work queues with retries;
- matching and coalition formation;
- agenda and document revision;
- Delphi and synthesis workflows;
- tiered planning;
- incident and disaster response.

These validate derived assignments, resumption, phase completion, and cross-stage provenance.

17.6 Phase 6: remove legacy syntax

After all examples and remote execution use the new representation:

- make string runtime references errors;
- remove `SharedState(scope=...)` routing behavior;
- remove schedule strings;
- remove implicit question-object-to-answer coercion;
- retain readers for old serialized packages behind a format-version adapter.

Chapter 18

Test specification

18.1 Unit tests

- Key validation and open/closed lifecycle.
- Mapping lookup, iteration, and immutability.
- Isolation of primitive contents between keys.
- Shared configuration and store binding.
- Runtime-reference construction, equality, hashing, and serialization.
- Answer dependency ordering and type compatibility.
- `Stem` placeholder and visibility validation.
- Schedule serialization and preflight.
- Idempotent before-question and after-answer operations.
- Store fingerprint mismatch detection.

18.2 Concurrency tests

- Concurrent writes to different keys.
- Conflicting writes to the same primitive.
- Repeated idempotency keys.
- Close racing with an action.
- Simultaneous-round reads at one watermark.
- Remote retries after a successful write but before acknowledgement.

18.3 Serialization tests

- Round trip one-state configuration.
- Round trip state map and router.
- Round trip every primitive used by the examples.
- Round trip all schedule types.
- Reject arbitrary callables during remote preflight.
- Reconstruct a job in a clean process without importing its example module.

18.4 Example equivalence tests

For every migrated example, compare legacy and redesigned versions on a deterministic test model:

- same participating agents and assignments;
- equivalent logical action sequence;
- equivalent final primitive record;
- equivalent visibility at each decision;
- no dependence on wall-clock completion order unless declared;
- successful serialization and clean-process execution.

Chapter 19

Acceptance criteria

The redesign is complete when:

1. `SharedState` unambiguously represents one keyed world.
2. `SharedStateMap` behaves as a read-only, lazily materialized mapping of keys to `SharedState` instances.
3. Scope routing occurs through an explicit `StateRouter` at run construction.
4. No new example uses Jinja strings for runtime values outside compiled `Stem` compatibility output.
5. No new example passes a question object where an answer reference is meant.
6. No new example uses a string interview schedule.
7. Every current example has a clearer redesigned equivalent.
8. Local and remote stores pass the same behavioral contract tests.
9. Jobs serialize and execute in a clean remote process.
10. Errors involving keys, routing, visibility, or scheduling occur before model calls whenever their cause is statically knowable.

Chapter 20

Decisions and open questions

20.1 Decisions made by this specification

- One `SharedState` equals one concrete key.
- `SharedStateMap` owns homogeneous configuration for many keys.
- Keys are created lazily by concrete `__getitem__` access; non-indexing reads do not create them.
- Routing is configured on the run with `.by(...)`.
- `__getitem__` accepts only concrete strings.
- Map deletion and cross-key transactions are excluded from version 1.
- `current`, `Question.answer`, `Stem`, and class schedules are the preferred authoring syntax.

20.2 Questions to resolve during prototyping

1. Should the public term be `key`, `scope_id`, or `scope` on a one-world `SharedState`? This specification uses `key` to avoid inheriting the current ambiguity.
2. Should `states.create(key)` be provided as a more explicit synonym for creation by indexing?
3. Should a literal-key single state be syntactic sugar for a one-entry map, or a distinct implementation sharing only protocols?
4. Where should `StateRouter` attach: `shared_state=states.by(...)`, a dedicated `state_router=` argument, or the schedule? This specification keeps it with `shared_state` because routing and ordering are independent.
5. How should a run package export a remote store snapshot without duplicating a large event history?
6. Which assignment schemas are required for static validation of `current.assignment.*`?

These questions can change surface details. They should not reverse the central separation between one shared state and a mapping of shared states.